

TRƯỜNG ĐẠI HỌC SƯ PHẠM – ĐẠI HỌC ĐÀ NẴNG
KHOA TOÁN – TIN

Học phần: Lập trình mạng (Network Programming) – Học kỳ I, năm học 2026 – 2027

BÀI TẬP THỰC HÀNH NHÓM SỐ 4

Chương 4 – Lập trình ứng dụng không hướng kết nối UDP

Chuyển ứng dụng chat của nhóm sang UDP: server một luồng cho cả phòng, client biết hẹn giờ – gửi lại, và thí nghiệm các lỗi thường gặp của UDP

Hình thức	Làm việc nhóm 5 sinh viên — giữ nguyên nhóm đã đăng ký ở Bài tập nhóm số 1. Bài làm chuyển giao thức chat và mô-đun protocol.py của nhóm ở Bài tập nhóm số 3 từ TCP sang UDP.
Thời hạn	Nộp trên e-Learning (nhhai.net) trước 21:00 thứ Năm, ngày 08/10/2026.
Sản phẩm nộp	Một báo cáo PDF và mã nguồn Python (không phải làm slide, không thuyết trình). Tên file bắt buộc phải bắt đầu với NhomZZ, trong đó ZZ là mã nhóm. Ví dụ: Nhom05_Baitap4.pdf
Chuẩn đầu ra	CLO2 – giải thích và vận dụng được lập trình ứng dụng không hướng kết nối UDP: datagram, server một socket phục vụ nhiều client, client có thời gian chờ và gửi lại, broadcast. CLO3 – chỉ ra được các lỗi thường xảy ra khi lập trình với UDP và cách hạn chế các lỗi này. CLO5 – tổ chức làm việc nhóm, tìm kiếm tài liệu tham khảo có giá trị. CLO7 – có ý thức đóng góp để tạo ra một chương trình hoàn chỉnh.
Công cụ	Python 3.12 trở lên (không cài thêm thư viện ngoài); Visual Studio Code (extension Python) hoặc PyCharm Community; nc hoặc ncst (chế độ -u); netstat hoặc lsof.

1. Mục tiêu

Sau khi hoàn thành bài tập, sinh viên có thể:

- Xây dựng được UDP server một luồng phục vụ cả phòng chat: nhận diện client theo địa chỉ, đẩy tin tức thì, dọn client im lặng, không dừng vì datagram hỏng.
- Viết được UDP client có thời gian chờ, gửi lại có giới hạn, luồng nhận chạy nền và nhịp tim; tìm được server trong mạng LAN bằng broadcast.
- Tái hiện, đo đạc bằng số liệu chạy thật và hạn chế được các lỗi thường gặp của UDP — mất gói, lặp gói, datagram quá lớn — thay vì chỉ kể tên (CLO3).
- Kiểm thử chương trình UDP trong điều kiện mạng xấu bằng giả lập mất gói, không chỉ trên 127.0.0.1 — nơi không bao giờ mất gói.

2. Nhiệm vụ

Quy ước chung: mỗi nhóm dùng cổng UDP **9000 + mã nhóm** (nhóm 05 dùng 9005; được trùng số với server TCP ở Bài tập 3 vì TCP và UDP có hai không gian cổng

riêng). Mọi chương trình, kể cả `test_udp_server.py` và `exp_udp.py`, phải nhận `--host` và `--port` bằng `argparse`; mặc định server là `0.0.0.0`, các chương trình còn lại là `127.0.0.1`. Trên `127.0.0.1` không bao giờ mất gói, nên server phải có `--loss` để giả lập mất gói; giảng viên chạy thử mã của nhóm cả khi có và khi không có mất gói.

2.1. Phần A – Server phòng chat UDP `udp_chat_server.py` (3,0 điểm)

Nhóm chuyển đặc tả giao thức chat của mình sang UDP rồi cài đặt server một luồng phục vụ cả phòng: LOGIN, MSG, WHO, PING, QUIT, DISCOVER và hai loại tin server chủ động đẩy tới là FROM và SYS. Khung mã có trong slide Chương 4, mục 4.1 – 4.2.

1. **Bản đặc tả giao thức trên UDP.** Cập nhật bảng đặc tả theo mẫu ở Bảng 1 cho đủ các lệnh trên, kèm bảng mã lỗi ít nhất năm mã và sơ đồ trạng thái phiên có thêm cạnh "hết giờ". Ghi rõ: một datagram là một thông điệp; giới hạn kích thước (datagram $\leq 1\ 200$ byte, nội dung $\leq 1\ 000$ byte, tên ≤ 20 ký tự); chu kỳ nhịp tim; lệnh nào nhận hai lần vẫn vô hại.
2. **Tổ chức mã thành hàm.** Server tách thành `main()`, `create_socket()`, `serve_forever()`, `handle_datagram(text, addr)`, `deliver()` và `reap_idle()`. Hàm `handle_datagram()` không gọi `sendto()` mà trả về danh sách (thông điệp, địa chỉ) cần gửi, phần tử đầu luôn là trả lời cho người gửi. `send_dgram()` và `recv_dgram()` đặt trong `protocol.py` (MAX_DGRAM = 1200, nhận MAX_DGRAM + 1 byte để biết datagram bị cắt), server và client cùng import.
3. **Phiên theo địa chỉ và các lệnh.** Phiên lưu trong từ điển có khóa là (ip, cổng). LOGIN kiểm tra tên như Bài tập 3 và từ chối ký tự điều khiển; gửi lại từ cùng địa chỉ, cùng tên thì nhận OK như lần đầu. MSG|a11|... trả OK và đẩy ngay FROM|... tới mọi người khác; MSG|<tên>|... chỉ đẩy tới người đó hoặc trả ERR|5; nội dung quá 1 000 byte trả ERR|4. WHO trả danh sách tên, PING|<mã> trả PONG|<mã>, QUIT trả BYE và báo SYS cho cả phòng; MSG, WHO gửi khi chưa đăng nhập trả ERR|3.
4. **Bền vững, cấu hình và log.** Socket server có `settimeout(1.0)`; vòng nhận bắt OSError (kể cả WinError 10054) và ValueError (datagram quá lớn, UTF-8 hỏng): ghi log rồi bỏ qua, không bao giờ dừng vì dữ liệu từ mạng. Client im lặng quá `--idle` giây (mặc định 60) bị xóa, cả phòng nhận SYS|<tên> mất liên lạc. Tham số: `--host`, `--port`, `--idle`, `--loss` (mất datagram đến, log ghi [MẤT]) và `--reply-loss` (Phần C); logging có mốc thời gian; Ctrl+C dừng gọn.
5. **Chạy thử bằng nc -u.** Dùng `nc -u` (hoặc `ncat -u`) gõ tay một phiên LOGIN → MSG → WHO → PING → QUIT có ít nhất hai lệnh sai, trong lúc một `udp_chat_client.py` đang ở trong phòng để thấy tin được đẩy tới. Chụp màn hình các cửa sổ và chép log server vào báo cáo dưới dạng văn bản.

Bảng 1. Mẫu bảng đặc tả thông điệp trên UDP (dòng đầu là ví dụ, nhóm thay bằng đặc tả của mình)

Lệnh	Cú pháp và điều kiện	Trả lời người gửi (đúng / sai)	Tin đẩy tới người khác	Nhận hai lần thì sao?
LOGIN	LOGIN <tên> Chưa đăng nhập; tên 1 – 20 ký tự, không chứa " " hay ký tự điều khiển	OK Chào <tên> ERR 1, ERR 2, ERR 3	SYS <tên> vào phòng (cả phòng)	Cùng địa chỉ, cùng tên: trả OK như lần đầu — vô hại
...	...	...	...	...

2.2. Phần B – Client `udp_chat_client.py` và kiểm thử `test_udp_server.py` (3,0 điểm)

Client không bao giờ được treo vì chờ một datagram không tới. Khung mã có trong slide Chương 4, mục 4.3.

- Đăng nhập có hẹn giờ và gửi lại.** Client dùng socket UDP đã `connect()` tới server và hàm `ask()` trong `protocol.py` (chờ 0,5 giây, gửi lại tối đa 4 lần, mỗi lần chờ gấp đôi, chỉ nhận trả lời khớp yêu cầu); nhận ERR thì cho nhập lại tên. Bắt riêng `TimeoutError`, `ConnectionRefusedError`, `EOFError` / `KeyboardInterrupt`, mỗi trường hợp in một câu tiếng Việt. Lập bảng: tình huống – cách tạo ra – thông báo in ra.
- Luồng nhận chạy nền và nhip tim.** Sau khi đăng nhập, một luồng nền (`threading.Thread(daemon=True)`) là nơi duy nhất đọc socket: in tin FROM / SYS và gửi PING 20 giây một lần. Luồng chính đọc bàn phím: nội dung là gửi cả phòng, @tên nội dung là gửi riêng, /who, /quit; mọi cách thoát (/quit, Ctrl+C, Ctrl+D) đều gửi QUIT.
- Tìm server bằng broadcast.** Tham số `--find`: bật `SO_BROADCAST`, gửi DISCOVER tới <broadcast> và công của nhóm, gom mọi trả lời HERE | ... trong 2 giây rồi kết nối tới server đầu tiên tìm thấy. Chạy thử trên hai máy (mạng dây phòng máy hoặc Wi-Fi phát từ điện thoại — nhiều mạng Wi-Fi chặn broadcast) và lập bảng: máy – hệ điều hành – IPv4 – vai trò – dòng `--find` in ra.
- Bộ kiểm thử `test_udp_server.py`.** Ít nhất tám ca, trong đó ít nhất ba ca dữ liệu sai; bắt buộc có ca LOGIN gửi lại, ca nội dung quá 1 000 byte và ca gửi thẳng một datagram 1 300 byte bằng `sock.send()` (server phải bỏ qua mà không dừng: ca ngay sau vẫn có trả lời). Mỗi ca có hẹn giờ và gửi lại để vẫn đúng khi server chạy `--loss 0.3`. Chạy ba lần không mất gói và ba lần với `--loss 0.3`, ghi vào Bảng 2; ca nào SAI phải giải thích bằng log server.
- Ba client cùng lúc.** A, B, C cùng vào phòng (C dùng `--find`). A gửi cả phòng, B gửi riêng cho A, C gõ /who. Sau đó đóng hẳn cửa sổ terminal của C (không gõ /quit): ghi thời điểm đóng và thời điểm A, B thấy C mất liên lạc. So sánh trong 3 – 5 dòng với hai client ở Bài tập 3 (server tuần tự).

Bảng 2. Mẫu bảng ca kiểm thử của test_udp_server.py (dòng đầu là ví dụ)

TT	Mục đích	Lệnh gửi	Trả lời mong đợi	Ba lần chạy không mất gói	Ba lần chạy --loss 0.3
1	Chặn sai trình tự	MSG all chào	ERR 3 Chưa đăng nhập	Đạt / Đạt / Đạt	
2	LOGIN gửi lại	LOGIN nam (lần 2)	OK Chào nam		
...	...	...	...	...	...

2.3. Phần C – Các lỗi thường gặp của UDP: thí nghiệm và cách hạn chế (2,0 điểm)

Đo bằng số liệu chạy thật hai lỗi kinh điển của UDP và cách hạn chế, rồi đối chiếu cả bảy lỗi đã học với mã của nhóm (CLO3). Hai thí nghiệm đặt chung trong exp_udp.py, chọn bằng tham số dòng lệnh (ví dụ python exp_udp.py loss --port 9005 và python exp_udp.py dup --port 9005), **chỉ chạy trên 127.0.0.1** vì chúng gửi đôn hàng nghìn datagram.

- Mất gói và gửi lại (lỗi ①).** Phần loss của exp_udp.py gọi ask() 1 000 lần (PING|i → PONG|i) tới server chạy --loss p với p = 0,1; 0,3; 0,5, đếm số yêu cầu xong sau 1, 2, 3, 4 lần gửi và số thất bại (được dùng thời gian chờ đầu 0,02 giây như exp_ask.py ở mục 4.3). Mỗi lần gửi mất độc lập với xác suất p, nên theo lý thuyết có $N \cdot (1 - p) \cdot p^{k-1}$ yêu cầu xong ở lần thứ k và $N \cdot p^4$ yêu cầu thất bại. Ghi số đo và số lý thuyết vào Bảng 3, nhận xét vì sao hai số gần nhau mà không bằng nhau.
- Lập gói và mã yêu cầu (lỗi ②).** Thêm vào server tham số --reply-loss (giả lập mất trả lời cho người gửi, khung gợi ý bên dưới) và mã yêu cầu: client gán mã tăng dần vào lệnh (7|MSG|all|...), server trả lời kèm đúng mã đó (7|OK|...) và nhớ (mã, trả lời) cuối cùng của từng địa chỉ; gặp lại mã cũ thì chỉ gửi lại trả lời cũ. Phần dup của exp_udp.py dùng hai socket: A gửi 100 tin cả phòng bằng ask(), B đếm số tin FROM nhận được. Chạy với --reply-loss 0.3 ba lần không gán mã, ba lần có gán mã, ghi vào Bảng 4.

Khung gợi ý cho mục 2 — giả lập mất trả lời trong serve_forever():

```

out = handle_datagram(text, addr)  # out[0]: trả lời người gửi
if out and random.random() < reply_loss:
    logging.info("%s:%d << %s [MẤT TRẢ LỜI]", *addr, out[0][0])
    out = out[1:]  # tin đẩy tới người khác vẫn đi
deliver(sock, out)
# Không mã: mất OK -> A gửi lại -> B nhận tin 7 hai lần
# Có mã: A gửi 7|MSG|all|tin 7, S trả 7|OK|... và nhớ (7, trả lời);
#      gặp lại mã 7 thì chỉ gửi lại trả lời cũ, không đăng lại

```

- Bảy lỗi trong ứng dụng của nhóm.** Lập Bảng 5 cho đủ bảy lỗi đã học: lỗi đó có xảy ra với ứng dụng chat của nhóm không và khi nào; nhóm hạn chế bằng đoạn mã nào (tệp – hàm) hoặc vì sao chấp nhận không xử lý; kiểm chứng bằng ca kiểm thử hay thí nghiệm nào. Riêng lỗi ⑥, chạy exp_overflow.py (mục 4.4 của

slide) trên máy của một thành viên và ghi hệ điều hành, SO_RCVBUF mặc định và tỉ lệ mất ở hai lần thử.

Bảng 3. Mẫu bảng thí nghiệm mất gói và gửi lại (N = 1 000; mỗi ô ghi "đo được / lý thuyết"; số lý thuyết của p = 0,3 là ví dụ)

p (--loss)	Xong sau 1 lần	Xong sau 2 lần	Xong sau 3 lần	Xong sau 4 lần	Thất bại
0,1	... / ...	... / ...	... / ...	... / ...	... / ...
0,3	... / 700	... / 210	... / 63	... / 18,9	... / 8,1
0,5	... / ...	... / ...	... / ...	... / ...	... / ...

Bảng 4. Mẫu bảng thí nghiệm lặp gói (server --reply-loss 0.3, A gửi 100 tin; mỗi ô ghi ba lần chạy)

Mã yêu cầu	Số lần A phải gửi lại	Số tin A báo thất bại	Số tin FROM B nhận được	Số tin bị lặp ở B
Không	... / ... / ...	... / ... / ...	... / ... / ...	... / ... / ...
Có	... / ... / ...	... / ... / ...	... / ... / ...	... / ... / ...

Bảng 5. Mẫu bảng bảy lỗi của UDP trong ứng dụng của nhóm (dòng ⑤ là ví dụ)

Lỗi	Có xảy ra với ứng dụng của nhóm không? Khi nào?	Nhóm hạn chế bằng (tệp – hàm) hoặc lý do không xử lý	Kiểm chứng bằng
① Mất gói			
② Lặp gói			
③ Sai thứ tự			
④ Bị cắt			
⑤ Quá lớn	Có: người dùng dán một đoạn văn rất dài	protocol.py – send_dgram() chặn > 1 200 byte; server – do_msg() trả ERR 4	Ca 1 300 byte và ca nội dung quá 1 000 byte (Bảng 2)
⑥ Tràn bộ đệm nhận			
⑦ Nguồn không tin được			

2.4. Phần D – Câu hỏi thảo luận (1,0 điểm)

Nhóm thảo luận và trả lời ngắn gọn (mỗi câu 5 – 10 dòng) trong báo cáo, có dẫn chứng từ kết quả chạy thực tế của chính nhóm ở các phần trên:

Câu 1. Server phòng chat UDP chỉ có một luồng nhưng đẩy được tin cho cả phòng ngay lập tức, trong khi server TCP tuần tự ở Bài tập 3 bắt client thứ hai phải chờ. Vì sao? Thiết kế UDP này phải trả giá những gì (nêu ít nhất ba điều)? Dẫn chứng bằng quan sát ở mục 5 của Phần B.

Câu 2. Vì sao `ask()` tăng gấp đôi thời gian chờ sau mỗi lần gửi lại thay vì gửi lại đều đặn mỗi 0,5 giây? Nếu thời gian chờ nhỏ hơn thời gian khứ hồi (RTT) thật của mạng thì lệnh MSG gặp chuyện gì, khi có và khi không có mã yêu cầu? Dẫn chứng bằng Bảng 3 và Bảng 4.

Câu 3. Server gửi tin cho cả phòng bằng N datagram unicast cho N người. Có nên thay bằng một datagram multicast tới một nhóm 239.x.x.x không? Nêu hai lợi ích và ít nhất hai khó khăn (tin riêng, đăng nhập, mạng thật, biết ai đã nhận...). Có thể chạy `mcast_send.py` và `mcast_listen.py` ở mục 4.4 của slide để dẫn chứng.

3. Sản phẩm nộp và quy cách

- **Báo cáo:** một tệp PDF xuất từ Word / Google Docs (không nộp ảnh quét), đặt tên `NhomZZ_Baitap4.pdf`, trong đó ZZ là mã nhóm. Cấu trúc bắt buộc, dùng đúng tên mục: Trang bìa (số nhóm; họ tên và mã sinh viên của mọi thành viên; công nhóm sử dụng); Phần A (Bảng 1 đã điền, bảng mã lỗi, sơ đồ trạng thái, log server); Phần B (bảng tình huống lỗi của client, bảng máy dùng khi tìm server, Bảng 2 đã điền, quan sát ba client); Phần C (Bảng 3, Bảng 4, Bảng 5 đã điền, kèm nhận xét); Phần D; Công cụ AI đã dùng (nếu có); Tài liệu tham khảo; Phụ lục A – Bảng phân công. Mọi số liệu quan sát được phải ghi thành bảng hoặc câu văn; ảnh chụp chỉ để minh họa, cắt gọn phần cần xem, đọc được chữ và có chú thích (Hình 1, Hình 2...). Báo cáo nộp phải là bản hoàn chỉnh, không còn ô trống hay ghi chú chờ điền của mẫu.
- **Mã nguồn:** năm tệp `protocol.py`, `udp_chat_server.py`, `udp_chat_client.py`, `test_udp_server.py`, `exp_udp.py` đặt ở thư mục gốc của tệp nén `NhomZZ_Baitap4_code.zip`; ảnh chụp đặt trong thư mục `anh/` và đặt tên theo mục (ví dụ `A5_nc.png`, `B3_find.png`, `C1_p03.png`); không nén `.venv`, `__pycache__`. Mỗi tệp mã có chú thích đầu tệp ghi tên nhóm, thành viên viết tệp đó, lệnh chạy, và phần nào lấy từ slide, phần nào nhóm tự viết thêm.
- **Chạy được bằng đúng lệnh ghi ở đầu tệp:** giảng viên chạy thử trên một máy với `127.0.0.1` và cổng của nhóm, bằng Python 3.12 trở lên, không cài thêm thư viện. Ví dụ: `python udp_chat_server.py --port 9005 --loss 0.3`, `python udp_chat_client.py --port 9005` và `python test_udp_server.py --port 9005`. Mã không chạy được bằng lệnh đã ghi bị tính là chưa hoàn thành phần tương ứng.
- **Thành viên:** danh sách ở trang bìa và Phụ lục A phải khớp với nhóm trên e-Learning tại thời điểm nộp; nhóm có thay đổi thành viên sau khi nộp phải nộp lại báo cáo trước hạn.
- **Cách chấm:** không phải làm slide và không thuyết trình. Giảng viên chấm trên báo cáo PDF theo rubric ở mục 4, đối chiếu với ảnh trong tệp nén, chạy thử mã nguồn của nhóm và có thể hỏi bất kỳ thành viên nào tại buổi thực hành.

- **Nơi nộp:** mục "Thực hành 4" trên e-Learning (nhhai.net); nhóm trưởng nộp một lần cho cả nhóm; nộp lại thì thay cả hai tệp.

4. Tiêu chí đánh giá (Rubric R1.2)

Tiêu chí	Điểm	Tốt (80 – 100%)	Đạt (50 – 79%)	Chưa đạt (< 50%)
A. Server phòng chat UDP	3,0	Đủ các lệnh đúng đặc tả; đầy tin tức thì cho cả phòng; LOGIN gửi lại vô hại; dọn client im lặng; có tham số, log, giả lập mất gói; không dừng vì datagram hỏng.	Chạy được LOGIN và MSG cho cả phòng; còn thiếu WHO, PING, dọn client im lặng hoặc giả lập mất gói.	Không chạy được bằng lệnh đã ghi, không theo đặc tả, hoặc dừng khi gặp datagram hỏng.
B. Client, tìm server và bộ kiểm thử	3,0	Client có hẹn giờ – gửi lại, luồng nhận nền, nhịp tim, báo lỗi rõ ràng, tìm được server bằng broadcast; bộ kiểm thử đủ ca, đúng cả khi --loss 0.3.	Client chat được khi mạng tốt; bộ kiểm thử không gửi lại hoặc thiếu ca bắt buộc.	Client treo khi mất một gói; không có bộ kiểm thử chạy được.
C. Các lỗi thường gặp của UDP	2,0	Đủ số liệu ba mức mất gói, so sánh đúng với lý thuyết; tái hiện được lặp gói và khắc phục bằng mã yêu cầu; bảng bảy lỗi chỉ đúng mã của nhóm.	Có thí nghiệm nhưng thiếu mức đo, thiếu so sánh lý thuyết hoặc chưa khắc phục được lặp gói.	Không có số liệu chạy thật; bảng bảy lỗi chỉ chép lại slide.
D. Câu hỏi thảo luận	1,0	Trả lời đúng cả ba câu, có lập luận và dẫn chứng từ kết quả chạy của nhóm.	Trả lời đúng hướng nhưng còn chung chung hoặc thiếu một câu.	Trả lời sai hoặc bỏ trống từ hai câu trở lên.
E. Hình thức báo cáo, mã nguồn và tài liệu tham khảo	1,0	Trình bày đúng quy cách; mã nguồn sạch, có chú thích; trích dẫn ít nhất ba tài liệu tin cậy.	Đúng quy cách; mã nguồn còn lộn xộn hoặc trích dẫn chưa đúng chuẩn.	Sai quy cách; mã nguồn không chú thích; không có tài liệu tham khảo.

5. Quy định

- Báo cáo hoặc mã nguồn sao chép giữa các nhóm, hoặc sao chép từ nguồn khác mà không trích dẫn, nhận 0 điểm cho tất cả các nhóm liên quan.
- Được phép dùng công cụ AI để hỗ trợ tra cứu, sửa lỗi, nhưng phải ghi rõ trong mục "Công cụ AI đã dùng" của báo cáo: công cụ nào, dùng cho việc gì; không dán ảnh chụp giao diện của công cụ AI vào báo cáo; mọi thành viên phải hiểu và giải thích được nội dung nộp khi giảng viên hỏi.

- Thành viên không tham gia (theo bảng phân công có xác nhận của nhóm) hoặc không có tên trong báo cáo nhận 0 điểm bài tập này.
- Sau khi chấm, giảng viên có thể yêu cầu nhóm demo trực tiếp sản phẩm và trả lời câu hỏi về phần việc của từng thành viên; điểm có thể được điều chỉnh tăng hoặc giảm theo kết quả demo, cho cả nhóm hoặc riêng từng thành viên.
- Chỉ chạy chương trình trong mạng của phòng thực hành và trên máy của chính mình. Không quét cổng, không gửi datagram tới máy của nhóm khác khi chưa được đồng ý. Broadcast chỉ dùng để tìm server (vài datagram mỗi lần); các thí nghiệm gửi đôn ở Phần C chỉ chạy trên 127.0.0.1.

6. Tài liệu tham khảo và gợi ý

- Slide bài giảng Chương 4 – Lập trình ứng dụng không hướng kết nối UDP (mục 4.1 cho giao thức chat trên UDP, mục 4.2 cho khung server, mục 4.3 cho khung client, ask() và UDP Pinger, mục 4.4 cho broadcast, multicast, bảy lỗi thường gặp và bảng kiểm trước khi nộp bài).
- B. Rhodes, J. Goerzen – Foundations of Python Network Programming, 3rd ed., Apress, 2014: Chương 2 (UDP).
- J. Kurose, K. Ross – Computer Networking: A Top-Down Approach, 8th ed., Pearson, 2021: mục 3.3 (Connectionless Transport: UDP) và bài thực hành UDP Pinger (Socket Programming Assignment 2).
- Python Software Foundation – The Python Standard Library: socket, threading, argparse, logging – <https://docs.python.org/3/library/socket.html>
- RFC 768 – User Datagram Protocol – <https://www.rfc-editor.org/rfc/rfc768>
- RFC 8085 – UDP Usage Guidelines (thời gian chờ, gửi lại, kích thước datagram, kiểm soát tốc độ gửi) – <https://www.rfc-editor.org/rfc/rfc8085>

Phụ lục A – Mẫu bảng phân công công việc và tự đánh giá

Ghi đủ mọi thành viên của nhóm. Cột "Công việc đảm nhận" ghi cụ thể phần đã làm (ví dụ: Phần A – handle_datagram và bảng mã lỗi; Phần C – exp_udp.py; tổng hợp báo cáo). Cột "Đóng góp (%)" do cả nhóm thống nhất — làm ngang nhau thì ghi bằng nhau (nhóm 5 người: 20 % mỗi bạn); nhóm chịu trách nhiệm về bảng này khi giảng viên hỏi.

TT	Họ và tên	Mã sinh viên	Công việc đảm nhận	Đóng góp (%)
1			Nhóm trưởng; ...	
2				
3				
4				
5				

